AI ASSISTED CODING

LAB-15.5

B.SUJAL REDDY

2303A51713

Batch-11

Task 1 – Student Attendance API

Task:

Create a RESTful API for student attendance tracking.

Instructions:

- **Endpoints required: o GET /attendance → View attendance records o POST /attendance → Mark attendance o PUT /attendance/{id} → Update attendance record o DELETE /attendance/{id} → Remove attendance entry**
- **Ensure JSON-based request and response handling.**

Expected Output:

- **API for maintaining and updating attendance records.**

PROMPT:

"""Generate a Python Flask REST API for student attendance with in-memory storage.

Implement GET /attendance, POST /attendance, PUT /attendance/<id>, DELETE /attendance/<id>.

Use JSON request/response and add comments."""

GIVEN CODE:

```python
"""Generate a Python Flask REST API for student attendance with in-memory storage.
Implement GET /attendance, POST /attendance, PUT /attendance/<id>, DELETE /attendance/<id>.
Use JSON request/response and add comments."""
from flask import Flask, request, jsonify
app = Flask(__name__)
# In-memory storage for attendance records
attendance_records = []
# GET /attendance - Retrieve all attendance records
@app.route('/attendance', methods=['GET'])
def get_attendance():
    return jsonify(attendance_records), 200
# POST /attendance - Create a new attendance record
@app.route('/attendance', methods=['POST'])
def create_attendance():
    data = request.get_json()
    if not data or 'student_name' not in data or 'date' not in data:
        return jsonify({'error': 'Invalid input'}), 400
    new_record = {
        'id': len(attendance_records) + 1,
        'student_name': data['student_name'],
        'date': data['date']
    }
    attendance_records.append(new_record)
    return jsonify(new_record), 201
# PUT /attendance/<id> - Update an existing attendance record
@app.route('/attendance/<int:record_id>', methods=['PUT'])
def update_attendance(record_id):
    data = request.get_json()
    if not data or 'student_name' not in data or 'date' not in data:
        return jsonify({'error': 'Invalid input'}), 400
    for record in attendance_records:
        if record['id'] == record_id:
            record['student_name'] = data['student_name']
            record['date'] = data['date']
            return jsonify(record), 200
    return jsonify({'error': 'Record not found'}), 404
# DELETE /attendance/<id> - Delete an attendance record
@app.route('/attendance/<int:record_id>', methods=['DELETE'])
def delete_attendance(record_id):
    global attendance_records
    attendance_records = [record for record in attendance_records if record['id'] != record_id]
    return jsonify({'message': 'Record deleted'}), 200
if __name__ == '__main__':
    app.run(debug=True)
```

Task 2 – Inventory Management API

Task:

Develop a REST API for warehouse inventory management.

Instructions:

• Endpoints required:

o GET /inventory → List all items o POST

/inventory → Add new item o PUT

/inventory/{id} → Update stock quantity o

DELETE /inventory/{id} → Remove an item

• Implement validation for stock values.

Expected Output:

• API capable of managing inventory stock levels.

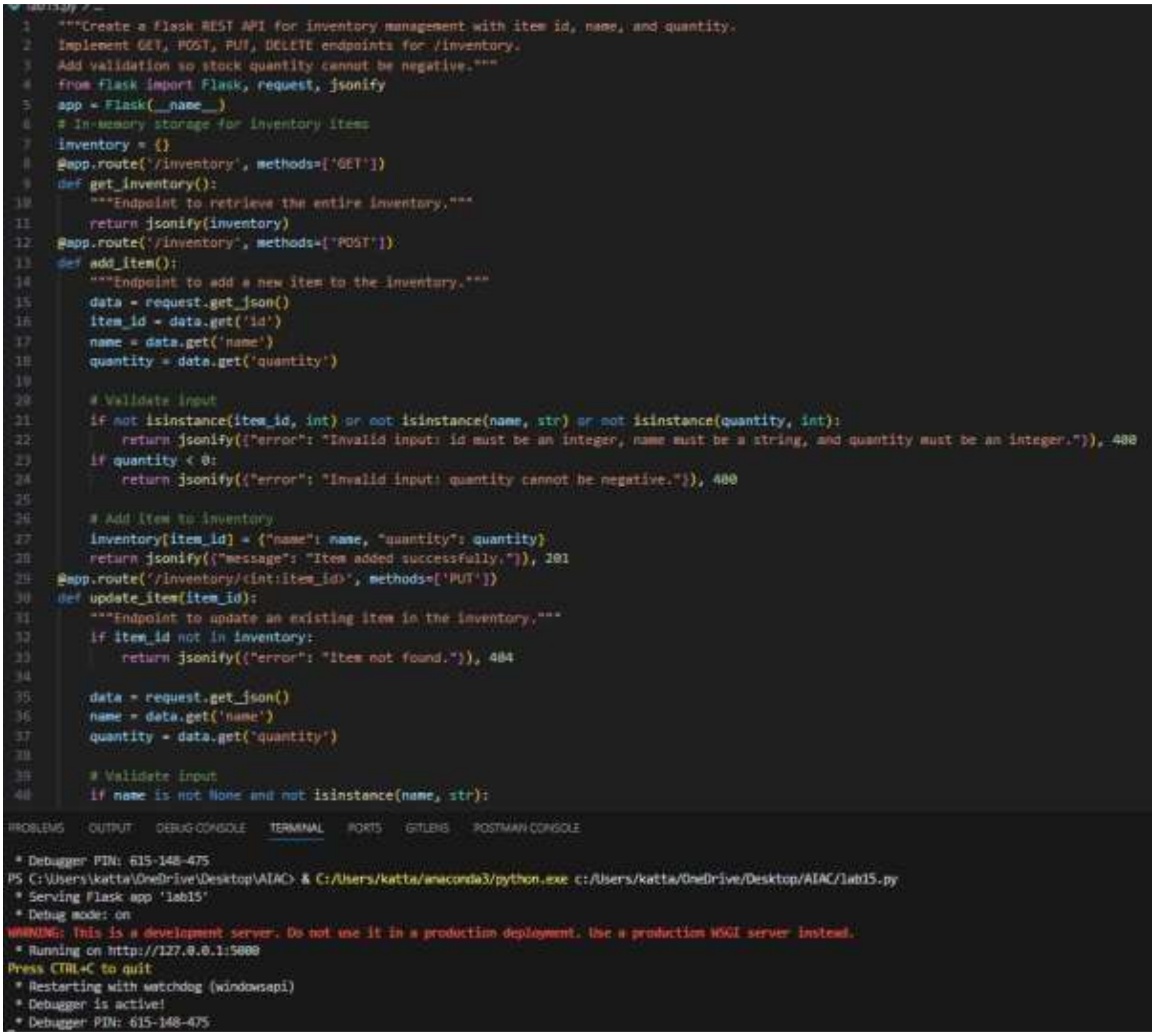
<u>PROMPT:</u>

"""Create a Flask REST API for inventory management with item id, name, and quantity.

Implement GET, POST, PUT, DELETE endpoints for /inventory.

Add validation so stock quantity cannot be negative.""" **<u>GIVEN</u>**

<u>CODE AND OUTPUT:</u>



```python
"""Create a Flask REST API for inventory management with item id, name, and quantity.
Implement GET, POST, PUT, DELETE endpoints for /inventory.
Add validation so stock quantity cannot be negative."""
from flask import Flask, request, jsonify
app = Flask(__name__)
# In-memory storage for inventory items
inventory = {}
@app.route('/inventory', methods=['GET'])
def get_inventory():
    """Endpoint to retrieve the entire inventory."""
    return jsonify(inventory)
@app.route('/inventory', methods=['POST'])
def add_item():
    """Endpoint to add a new item to the inventory."""
    data = request.get_json()
    item_id = data.get('id')
    name = data.get('name')
    quantity = data.get('quantity')

    # Validate input
    if not isinstance(item_id, int) or not isinstance(name, str) or not isinstance(quantity, int):
        return jsonify({"error": "Invalid input: id must be an integer, name must be a string, and quantity must be an integer."}), 400
    if quantity < 0:
        return jsonify({"error": "Invalid input: quantity cannot be negative."}), 400

    # Add item to inventory
    inventory[item_id] = {"name": name, "quantity": quantity}
    return jsonify({"message": "Item added successfully."}), 201
@app.route('/inventory/<int:item_id>', methods=['PUT'])
def update_item(item_id):
    """Endpoint to update an existing item in the inventory."""
    if item_id not in inventory:
        return jsonify({"error": "Item not found."}), 404

    data = request.get_json()
    name = data.get('name')
    quantity = data.get('quantity')

    # Validate input
    if name is not None and not isinstance(name, str):
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   GITLENS   POSTMAN CONSOLE

 * Debugger PIN: 615-148-475
PS C:\Users\katta\OneDrive\Desktop\AIAC> & C:/Users/katta/anaconda3/python.exe c:/Users/katta/OneDrive/Desktop/AIAC/lab15.py
 * Serving Flask app 'lab15'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with watchdog (windowsapi)
 * Debugger is active!
 * Debugger PIN: 615-148-475
```

<u>Task 3 – Hotel Room Booking API</u>

Task:

Develop a RESTful API for hotel room booking.

Instructions:

• Endpoints required:

o GET /rooms → View available rooms o POST /rooms/book → Book a room o GET /rooms/{id} → View booking details o DELETE /rooms/{id} → Cancel booking • Implement error handling for unavailable rooms.

Expected Output:

• API simulating hotel room booking operations.

<u>PROMPT:</u>

"""Generate a Flask REST API for hotel room booking using in-memory data.

Endpoints: GET /rooms, POST /rooms/book, GET /rooms/<id>, DELETE /rooms/<id>.

Add error handling if a room is already booked."""

<u>GIVEN OUTPUT:</u>

```python
"""Generate a Flask REST API for hotel room booking using in-memory data.
Endpoints: GET /rooms, POST /rooms/book, GET /rooms/<id>, DELETE /rooms/<id>.
Add error handling if a room is already booked."""
from flask import Flask, request, jsonify
app = Flask(__name__)
# In-memory storage for rooms
rooms = {
    1: {"type": "single", "price": 100, "booked": False},
    2: {"type": "double", "price": 150, "booked": False},
    3: {"type": "suite", "price": 300, "booked": False}
}
@app.route('/rooms', methods=['GET'])
def get_rooms():
    """Endpoint to get the list of all rooms with their details."""
    return jsonify(rooms)
@app.route('/rooms/book', methods=['POST'])
def book_room():
    """Endpoint to book a room. Expects JSON with 'id' of the room to be booked."""
    data = request.get_json()
    room_id = data.get('id')
    if room_id not in rooms:
        return jsonify({"error": "Room not found"}), 404
    if rooms[room_id]['booked']:
        return jsonify({"error": "Room already booked"}), 400
    rooms[room_id]['booked'] = True
    return jsonify({"message": f"Room {room_id} booked successfully"})
@app.route('/rooms/<int:room_id>', methods=['GET'])
def get_room(room_id):
    """Endpoint to get details of a specific room by its ID."""
    if room_id not in rooms:
        return jsonify({"error": "Room not found"}), 404
    return jsonify(rooms[room_id])
@app.route('/rooms/<int:room_id>', methods=['DELETE'])
def delete_room(room_id):
    """Endpoint to delete a room by its ID. This will remove the room from the in-memory storage."""
    if room_id not in rooms:
        return jsonify({"error": "Room not found"}), 404
    del rooms[room_id]
    return jsonify({"message": f"Room {room_id} deleted successfully"})
if __name__ == '__main__':    app.run(debug=True)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS POSTMAN CONSOLE

```
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with watchdog (windowsapi)
 * Debugger is active!
 * Debugger PIN: 615-148-475
```

<u>Task 4 – Online Voting API</u>

Task:

Create a RESTful API for an online voting system.

Instructions:

• Endpoints required:

o GET /candidates → List candidates

o POST /vote → Cast a vote o GET

/results → View voting results

• Ensure one vote per user using in-memory validation.

Expected Output:

• API simulating a secure online voting process.

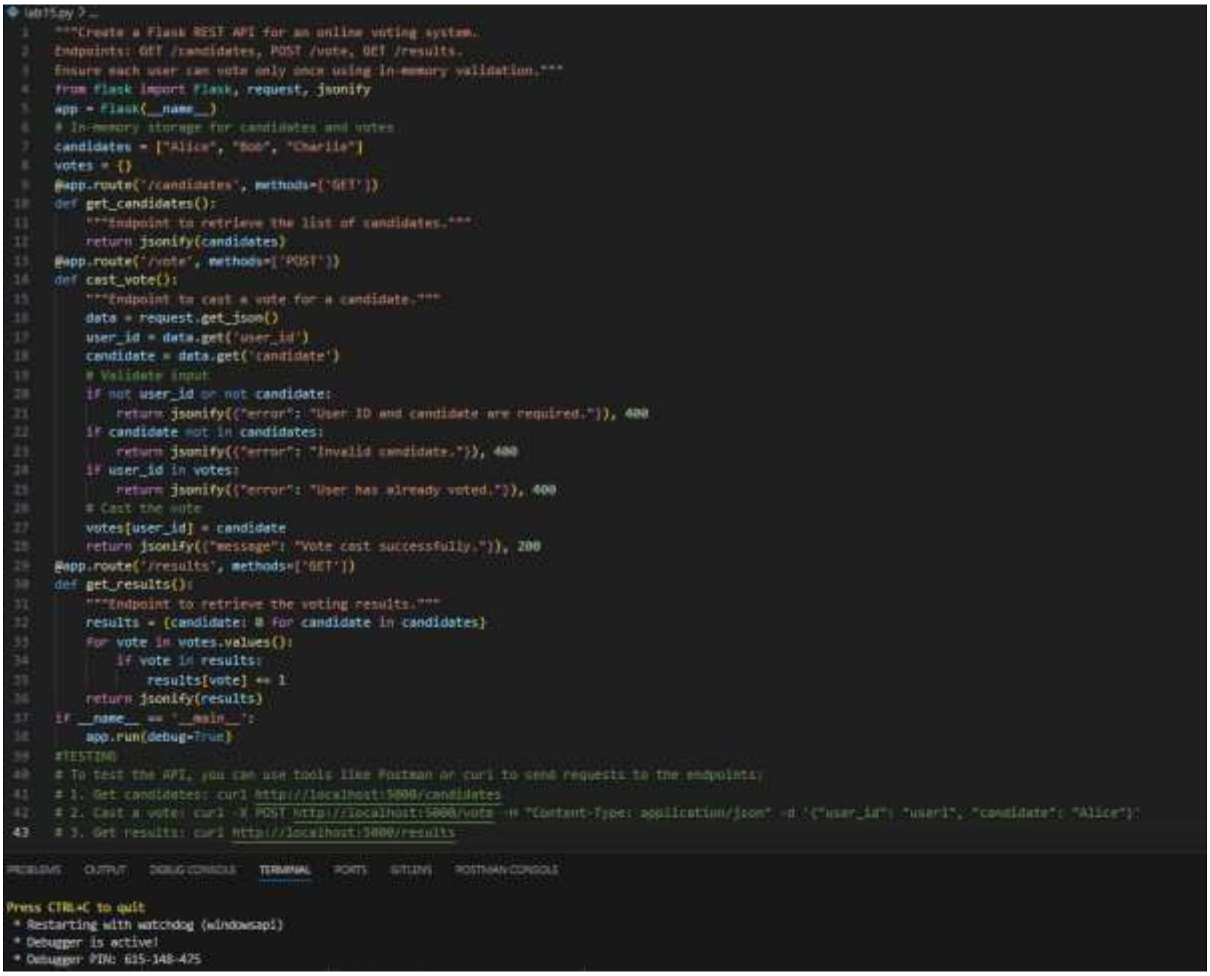
<u>PROMPT:</u>

"""Create a Flask REST API for an online voting system.

Endpoints: GET /candidates, POST /vote, GET /results.

Ensure each user can vote only once using in-memory validation.""" **<u>GIVEN</u>**

<u>OUTPUT:</u>



```python
"""Create a Flask REST API for an online voting system.
Endpoints: GET /candidates, POST /vote, GET /results.
Ensure each user can vote only once using in-memory validation."""
from flask import Flask, request, jsonify
app = Flask(__name__)
# In-memory storage for candidates and votes
candidates = ["Alice", "Bob", "Charlie"]
votes = {}
@app.route('/candidates', methods=['GET'])
def get_candidates():
    """Endpoint to retrieve the list of candidates."""
    return jsonify(candidates)
@app.route('/vote', methods=['POST'])
def cast_vote():
    """Endpoint to cast a vote for a candidate."""
    data = request.get_json()
    user_id = data.get('user_id')
    candidate = data.get('candidate')
    # Validate input
    if not user_id or not candidate:
        return jsonify({"error": "User ID and candidate are required."}), 400
    if candidate not in candidates:
        return jsonify({"error": "Invalid candidate."}), 400
    if user_id in votes:
        return jsonify({"error": "User has already voted."}), 400
    # Cast the vote
    votes[user_id] = candidate
    return jsonify({"message": "Vote cast successfully."}), 200
@app.route('/results', methods=['GET'])
def get_results():
    """Endpoint to retrieve the voting results."""
    results = {candidate: 0 for candidate in candidates}
    for vote in votes.values():
        if vote in results:
            results[vote] += 1
    return jsonify(results)
if __name__ == '__main__':
    app.run(debug=True)
#TESTING
# To test the API, you can use tools like Postman or curl to send requests to the endpoints:
# 1. Get candidates: curl http://localhost:5000/candidates
# 2. Cast a vote: curl -X POST http://localhost:5000/vote -H "Content-Type: application/json" -d '{"user_id": "user1", "candidate": "Alice"}'
# 3. Get results: curl http://localhost:5000/results
```

```
Press CTRL+C to quit
 * Restarting with watchdog (windowsapi)
 * Debugger is active!
 * Debugger PIN: 615-148-475
```

<u>Task 5 – Teacher Management API</u>

Task:

Develop a RESTful API to manage teacher records.

Instructions:

• Endpoints required:

o GET /teachers → List all teachers

o POST /teachers → Add a new teacher o PUT /teachers/{id} → Update teacher details o DELETE /teachers/{id} → Remove a teacher • Ensure proper error handling and JSON responses.

Expected Output:

• Functional API for managing teacher information.

PROMPT:

"""Generate a Flask REST API to manage teachers with id, name, and subject.

Implement GET, POST, PUT, DELETE endpoints for /teachers.

Return JSON responses and handle errors properly."""

GIVEN OUTPUT:

```python
"""Generate a Flask REST API to manage teachers with id, name, and subject.
Implement GET, POST, PUT, DELETE endpoints for /teachers.
Return JSON responses and handle errors properly."""
from flask import Flask, request, jsonify
app = Flask(__name__)
# In-memory storage for teachers
teachers = {}
@app.route('/teachers', methods=['GET'])
def get_teachers():
    """Endpoint to retrieve all teachers."""
    return jsonify(list(teachers.values())), 200
@app.route('/teachers', methods=['POST'])
def add_teacher():
    """Endpoint to add a new teacher."""
    data = request.get_json()
    if not data or 'name' not in data or 'subject' not in data:
        return jsonify({'error': 'Invalid input'}), 400
    teacher_id = len(teachers) + 1
    teacher = {
        'id': teacher_id,
        'name': data['name'],
        'subject': data['subject']
    }
    teachers[teacher_id] = teacher
    return jsonify(teacher), 201
@app.route('/teachers/<int:teacher_id>', methods=['PUT'])
def update_teacher(teacher_id):
    """Endpoint to update an existing teacher."""
    if teacher_id not in teachers:
        return jsonify({'error': 'Teacher not found'}), 404
    data = request.get_json()
    if not data or 'name' not in data or 'subject' not in data:
        return jsonify({'error': 'Invalid input'}), 400
    teacher = teachers[teacher_id]
    teacher['name'] = data['name']
    teacher['subject'] = data['subject']
    return jsonify(teacher), 200
@app.route('/teache (parameter) teacher_id: Any ['DELETE'])
def delete_teacher(teacher_id):
    """Endpoint to delete a teacher."""
    if teacher_id not in teachers:
        return jsonify({'error': 'Teacher not found'}), 404
    del teachers[teacher_id]
    return jsonify({'message': 'Teacher deleted'}), 200
```

BLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS POSTMAN CONSOLE

```
ss CTRL+C to quit
Restarting with watchdog (windowsapi)
Debugger is active!
Debugger PIN: 615-148-475
```